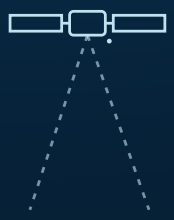
MARINE INTELLIGENCE PLATFORM

MarisAI

How it works, step by step

A plain-language story first, then the full technical walkthrough: data in, intelligence out. Each moving part is drawn as a diagram or a frame-by-frame storyboard.



Codebase: **VelvetGradient-26/MarisAI** @ `2f64b5f` (main, 25 Sep 2026)
Includes an infrastructure and logic audit of the EC2 deployment, carried out 19 Oct 2026
~115,000 lines across backend (Python/FastAPI), frontend (React/MapLibre) and offline ML

CONTENTS

What's inside

Part I needs no technical background. Part II starts from the same story and opens every box. Part III is the audit you asked for: deployment versions and the logic errors found, reported only and not fixed.

Part I - MarisAI in plain language

1 What MarisAI is, in one breath
2 The one rule: never fake a number
3 The journey of a single number
4 A fisher's morning, frame by frame
5 The six things you can do with it
6 What the colours and arrows on the map mean

Part II - Technical implementation, step by step

7 System architecture
8 Boot sequence and the scheduler
9 Data ingestion and caching
10 Lifecycle of one API request
11 The map and the GPU particle engine
12 Detectors: eddies, heatwaves, upwelling
13 The forecasting engine
14 The Ocean Assistant (multi-agent)
15 Grounding: how numbers are verified
16 Safety tools: risk, routing, geofence, alerts
17 Dashboard, downloader, point brief
18 Alert watches (email)
19 Offline machine learning
20 Deployment on AWS EC2
21 Technology stack reference

Part III - Audit report (19 Oct 2026)

22 Version check: EC2 vs local vs GitHub
23 Deployment and infrastructure findings
24 Business-logic findings
25 Test, lint and build results

About the "motion graphics". A PDF cannot play animation. Every moving process (particles drifting, an eddy being tracked, a route being searched, a question travelling through the agents) is drawn as a **storyboard**: numbered frames read left to right, with arrows for motion and time. If you need these as real video, they can be rendered to MP4 from the same diagrams.

MarisAI in plain language

No jargon. If you have ever checked a weather app before going outside, you already understand most of this.

CHAPTER 1

What MarisAI is, in one breath

MarisAI is a **weather app for the ocean, with a careful expert you can talk to**. It collects what satellites, buoys and ocean models measure every day, draws it on a globe you can spin, predicts what happens next, and answers questions like "Is it safe to go out?" or "Where might the fish be?" in your own language.

The ocean is measured constantly. European, American, Indian and other agencies publish sea temperature, wind, waves, currents, plankton and seafloor depth every few hours. That information is scattered across dozens of websites, in formats only specialists can open. A fisher in Kerala, a coast guard officer or a student has no realistic way to use it.

MarisAI does that work for them:

It gathers

Pulls live data from 14+ public sources on a schedule: Copernicus Marine, NOAA, NASA, Open-Meteo, Global Fishing Watch, GEBCO, IMD, GDACS, OBIS and more.

It understands

Turns raw numbers into meaning: "this water is unusually warm for October", "a swirl of current is forming here", "plankton is blooming".

It explains

Shows it on a map, in charts and in a chat assistant, and always says where every number came from.

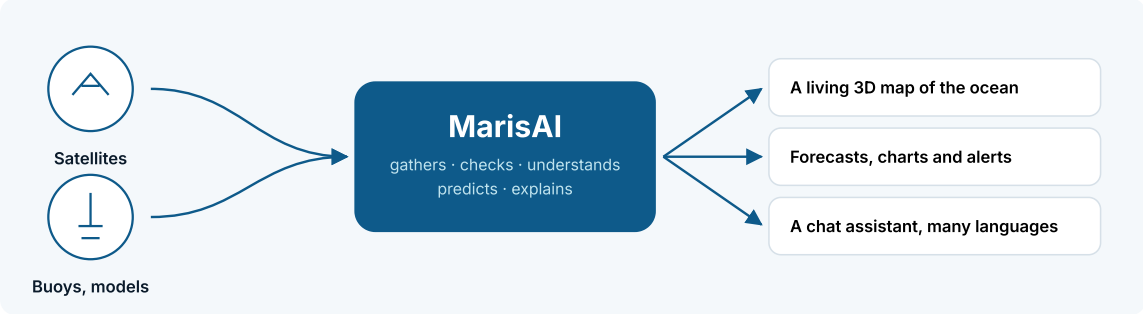


Figure 1. The whole idea in one picture: scattered measurements go in, one understandable picture comes out.

Who it is for

PERSON	THEIR QUESTION	WHERE MARISAI ANSWERS IT
Fisher	Is it safe tomorrow? Where are fish likely?	Assistant (risk verdict, fishing-zone screening, safe route)
Coastal authority	Any cyclone or severe-weather alert near our coast?	Assistant, dashboard alerts, email watches
Researcher / student	How has sea temperature changed here? Can I download it?	Metric pages, point analytics, bulk downloader
Ship operator	Which path avoids rough water, protected areas and the Sri Lanka boundary?	Assistant route planner

CHAPTER 2

The one rule: never fake a number

Most apps show **0** or a blank when data is missing. In the ocean, "0 mg/m³ of plankton" is not "nothing to report". It is a false reading that could send a boat to the wrong place. MarisAI is built around refusing to do that.

What a careless app shows

Chlorophyll near Kochi

0.00 mg/m³

Looks like a real reading. It isn't.

What MarisAI shows

Chlorophyll near Kochi

Not available right now

Reason: cloud cover blocked the satellite today.

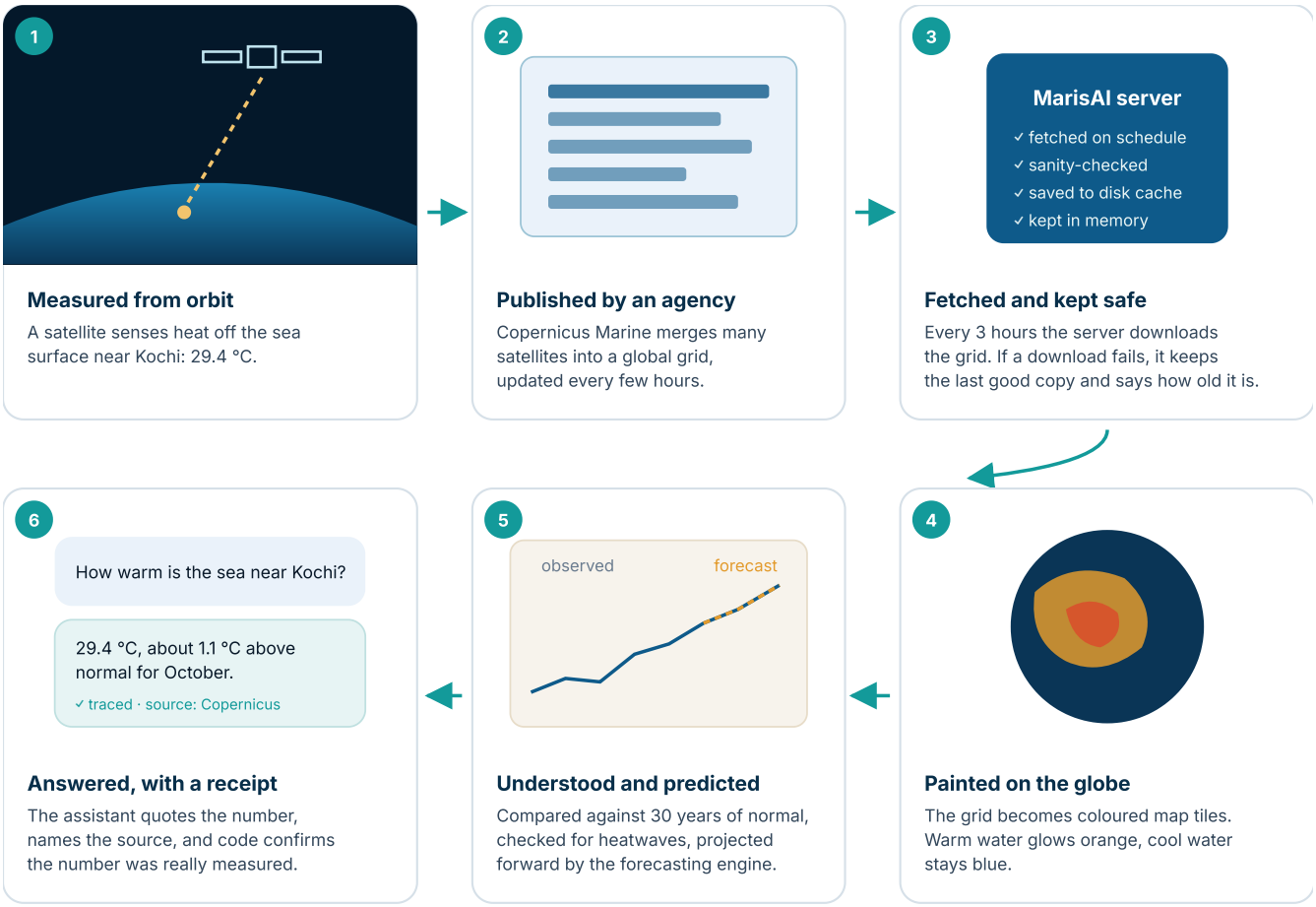
The same discipline runs all the way through:

1. **Every panel carries a flag**: available or not, and if not, a reason the screen actually displays.
2. **Predictions are labelled as predictions**, separately from observations, along with how good the model is.
3. **The assistant is fact-checked by code.** After it writes an answer, MarisAI scans every number in it and checks that number came from a real data tool. If one didn't, the answer is marked "not fully traced" instead of being shown as trustworthy.
4. **A forecast is only published if it beats "just assume tomorrow equals today".** If the model can't beat that simple guess, it is not shipped.

CHAPTER 3 · STORYBOARD

The journey of a single number

Follow one sea-temperature reading from space to your screen. Read the frames left to right, top to bottom.

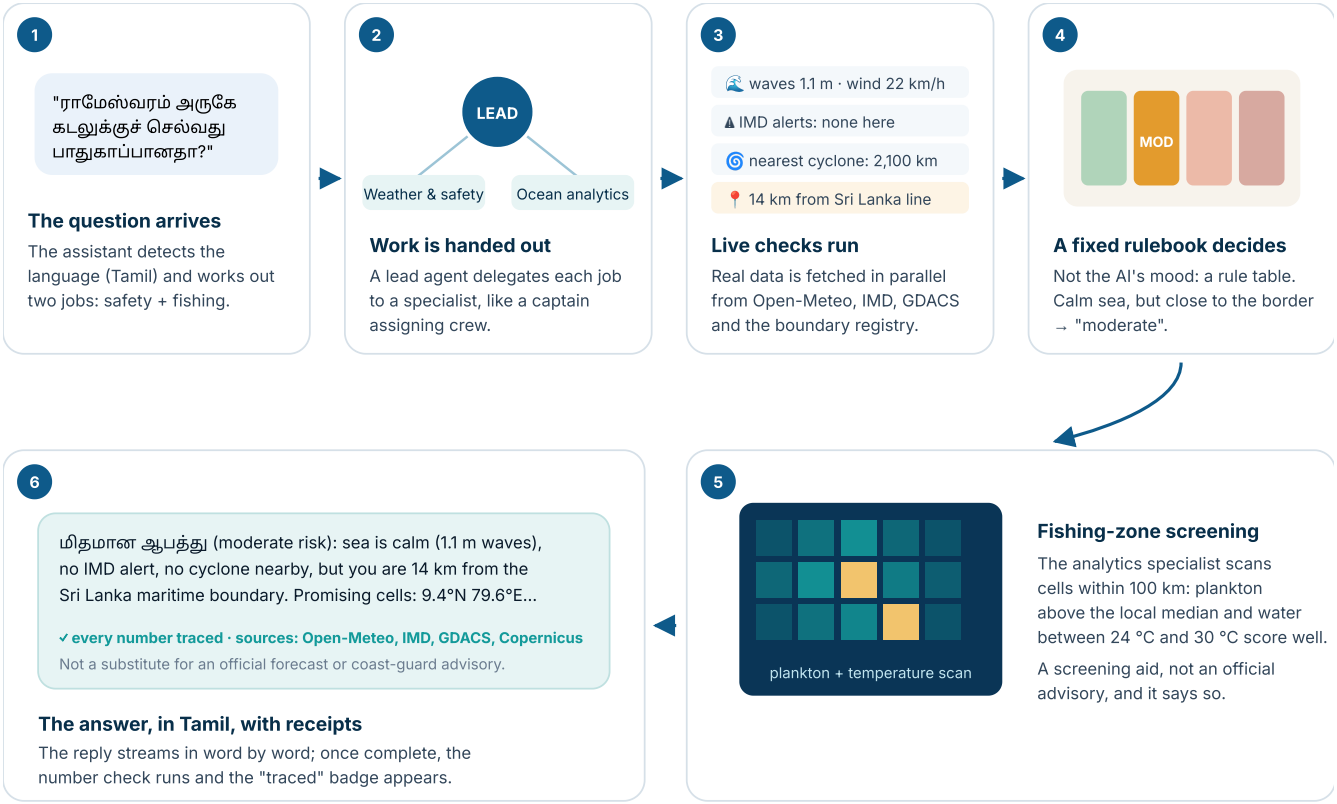


Storyboard A. One number, six frames. The same path is followed by wind, currents, waves, plankton and every other variable.

CHAPTER 4 · STORYBOARD

A fisher's morning, frame by frame

Ravi runs a small boat out of Rameswaram. At 5 a.m. he types, in Tamil: *"Is it safe to go out near Rameswaram? Where should I fish?"* Here is what happens in the next few seconds.



Storyboard B. Six frames, roughly 5 to 15 seconds of real time. Figures are illustrative. Part III notes that this risk verdict uses *current* conditions even when the question says "tomorrow".

CHAPTER 5

The six things you can do with it

🌍 Map /map

A spinnable 3D globe. Switch on layers: sea temperature, plankton, wind, currents (as flowing particles), waves, seafloor depth, fishing activity, live ships, swirling eddies, marine heatwaves, coastal upwelling, cyclones, protected areas, and model forecasts. Click anywhere for a full panel about that spot.

📊 Dashboard /dashboard

The ocean's vital signs right now: global averages, which data sources are healthy, active alerts, a written "Ocean Story" summary. Each metric opens a full page with history, forecast and what drives it.

📍 Point analytics & brief /analytics

Every metric at one coordinate, and a one-page "point brief" you can save as PDF: conditions, forecasts, nearby eddies, fishing habitat, recorded species.

💬 Assistant /assistant

Ask anything in English or Indian languages. Only here: safe-route planning, boundary checks, cyclone and IMD alerts, drift prediction, web and literature research.

⬇️ Downloader /download

Pick a region, a date range and up to 34 ocean variables from 14 providers; get CSV, JSON or PDF, aligned onto one timeline.

📚 Docs /docs

A searchable manual of how everything works, how it was tested and where it is weak.

Email watches. You can also ask MarisAI to watch a place for you. After you confirm by email, it checks every 15 minutes for IMD alerts, cyclones and algal-bloom risk, and emails you when something changes.

CHAPTER 6

What the colours and arrows on the map mean

LAYER	WHAT YOU SEE	WHAT IT MEANS IN PLAIN WORDS
Sea surface temperature	Blue to red colour wash	How warm the top of the sea is. Many fish prefer specific temperature bands.
Chlorophyll	Green intensity	How much microscopic plant life (plankton) is in the water, the base of the food chain.
Currents / wind / Stokes drift	Thousands of moving streaks	Which way water or air is moving, and how fast. Longer, brighter streaks mean faster.
Combined drift	Streaks	Where a floating object (or a person in the water) would actually be pushed: current + wave drift + a little wind.
Eddies	Rings, two colours	Giant slow whirlpools, tens to hundreds of km wide. They trap nutrients and fish.
Marine heatwaves	Orange to dark-red cells	Places where water has been unusually hot for its season for at least 5 days in a row.
Upwelling	Coastal bands	Wind pushing surface water away from shore, pulling cold nutrient-rich water up. Often good fishing.
Forecast layers	Raster with hatched areas	Model predictions. Hatched means "we can't forecast here", never painted as a fake value.
eDNA coverage	Sparse dots	Where scientists have sampled DNA from seawater. Mostly empty, and that emptiness is real.
EEZ / boundaries / protected areas	Lines and outlines	India's economic zone, the India-Sri Lanka boundary, and marine parks.

II

Technical implementation

The same story, with every box opened. Paths refer to the repository at commit `2f64b5f`.

CHAPTER 7

System architecture

Three runtimes: a React single-page app, a FastAPI backend that owns every external integration and cache, and an offline ML workspace whose outputs the backend only reads.

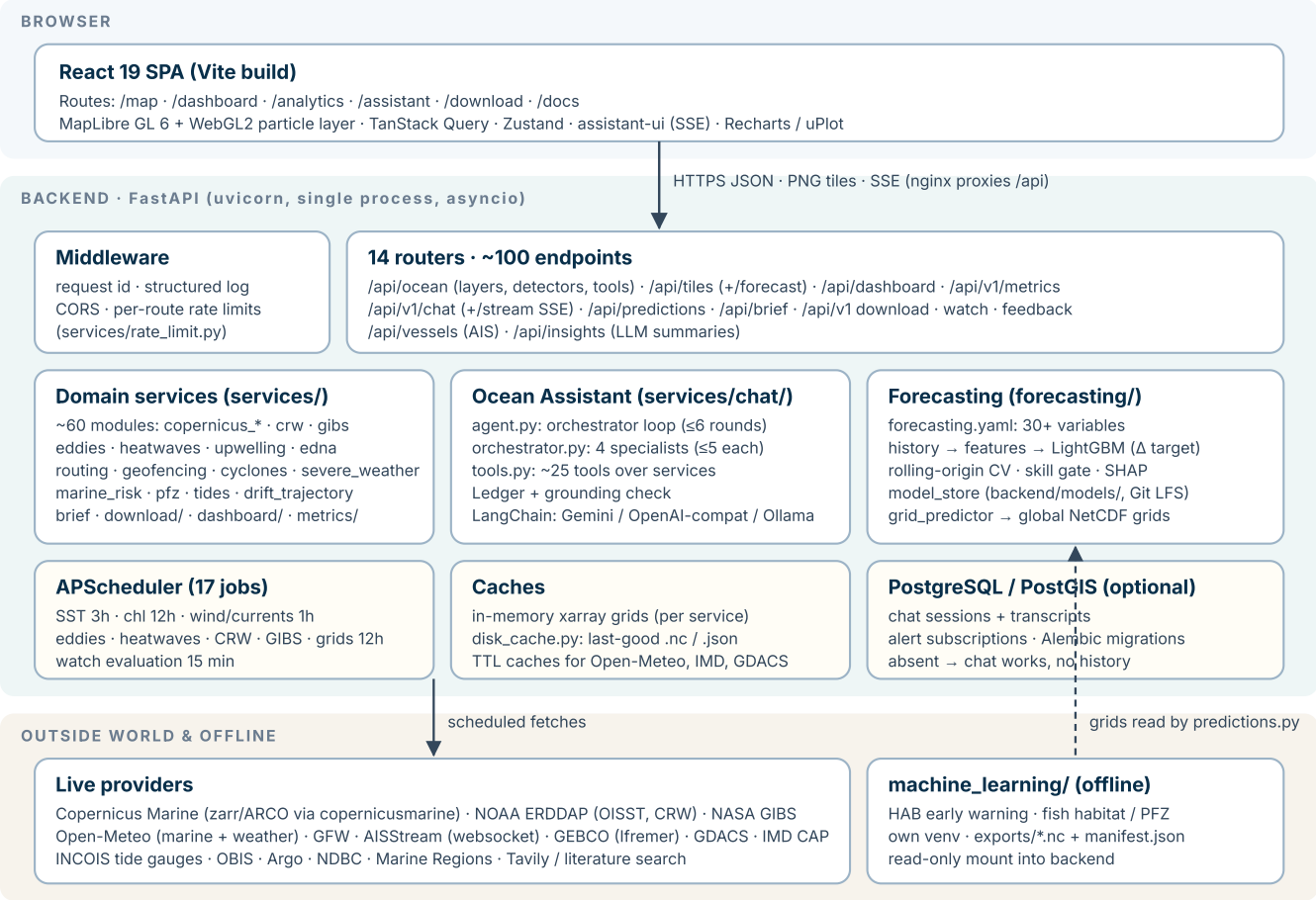


Figure 2. Component map. The browser never talks to a provider directly; every external call goes through a backend service that owns its cache, its failure handling and its availability flag.

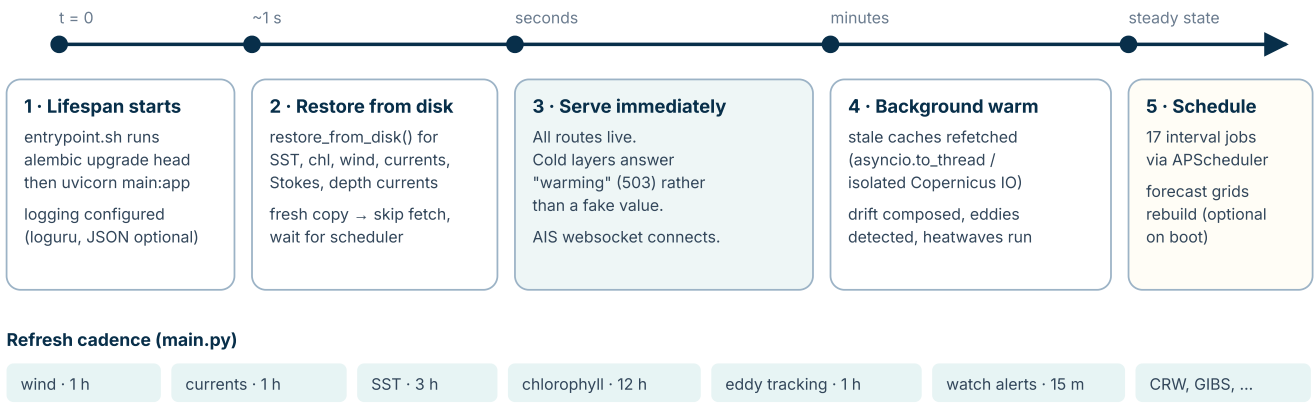
Design principles visible in the code

PRINCIPLE	WHERE IT LIVES
Availability, not placeholders. Every response carries `available` + `unavailable_reason`, or a three-way `ready / warming / unavailable` status.	dashboard services, KPI cards, layer endpoints
Cache, never fetch per request. Global grids are fetched by the scheduler; requests read memory.	`main.py` lifespan + `app/core/disk_cache.py`
Static config in code. Ports, boundaries, MPAs, provider registry, forecasting variables are code/YAML, not DB tables.	`services/ports.py`, `geofencing.py`, `download/registry.py`, `forecasting.yaml`
Each failure isolated. One dead upstream degrades one panel or one sub-check, never the whole answer.	`marine_risk.assess`, `asyncio.gather(..., return_exceptions=True)`

CHAPTER 8 · TIMELINE

Boot sequence and the scheduler

The server must answer within a second of starting, even though a full set of global grids takes minutes to download. It does this by restoring yesterday's copy from disk first, then refreshing in the background.



Timeline C. Boot. Observed on EC2 on 19 Oct 2026: containers healthy about 60 s after power-on, with SST/wind/currents restored from 20-22 Sep disk copies while fresh fetches ran.

CHAPTER 9 · FLOW

Data ingestion and caching

Copernicus is the backbone (SST, wind, currents, waves, biogeochemistry). Its client is heavy, slow and occasionally crashes, so it is fenced off.

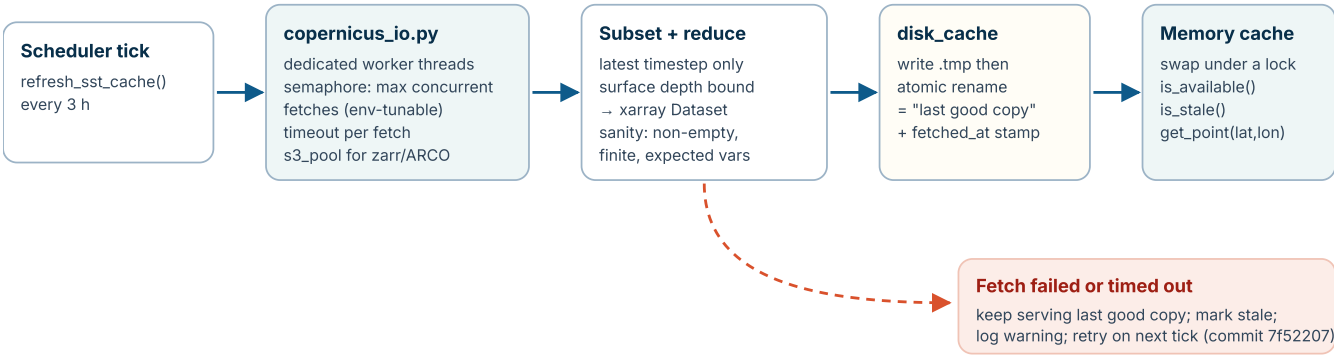


Figure 3. Ingestion. One fetch feeds many consumers, which is why PFZ screening can scan dozens of cells inside one chat turn: each is an array lookup, not a network call.

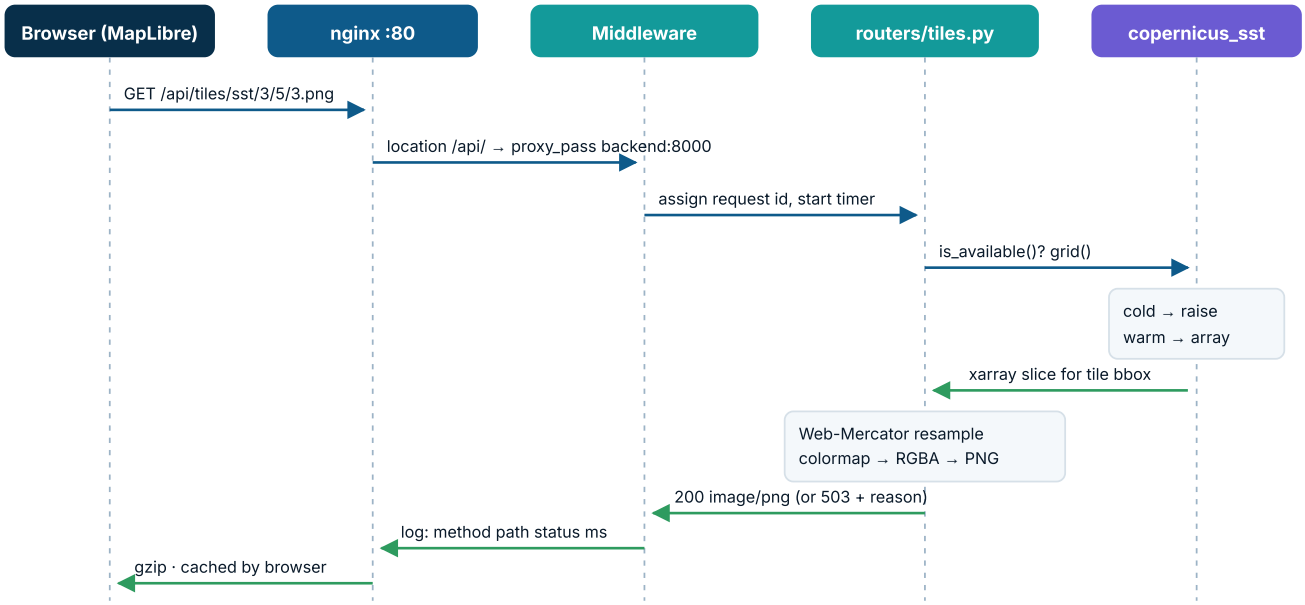
Provider families

FAMILY	PROVIDERS	ACCESS PATTERN
Gridded ocean physics & biogeochemistry	Copernicus Marine (analysis-forecast + reanalysis), NOAA OISST (ERDDAP), NOAA Coral Reef Watch	Scheduled global grid, disk + memory cache
Point weather & waves	Open-Meteo marine + forecast APIs	Per-request, 10 min TTL cache, batched up to 100 points
Imagery	NASA GIBS (11 tracked products)	Tile URLs proxied, capability cache
Human activity	Global Fishing Watch (effort), AISStream (live vessels)	Token API; persistent websocket
Hazards & advisories	GDACS (cyclones), IMD CAP 1.2 feed, INCOIS tide gauges	Keyless, 10-15 min TTL
Static geography	GEBCO 2021 (Ifremer ERDDAP), Marine Regions EEZ/IMBL	On demand / bundled JSON
Biology	OBIS (incl. eDNA), GBIF (ML only), Argo floats, NDBC buoys	On demand

CHAPTER 10 · SEQUENCE

Lifecycle of one API request

Example: the map asks for a sea-temperature tile at zoom 3.



Sequence D. A tile request. A cold cache answers 503 with a reason, which the layer panel renders as "warming", never as an empty or zero-valued tile.

> The same pattern (route → service → cache → availability-flagged JSON) repeats for every layer, KPI and detector endpoint. Only point-weather tools (Open-Meteo, IMD, GDACS) fetch on demand, behind short TTL caches.

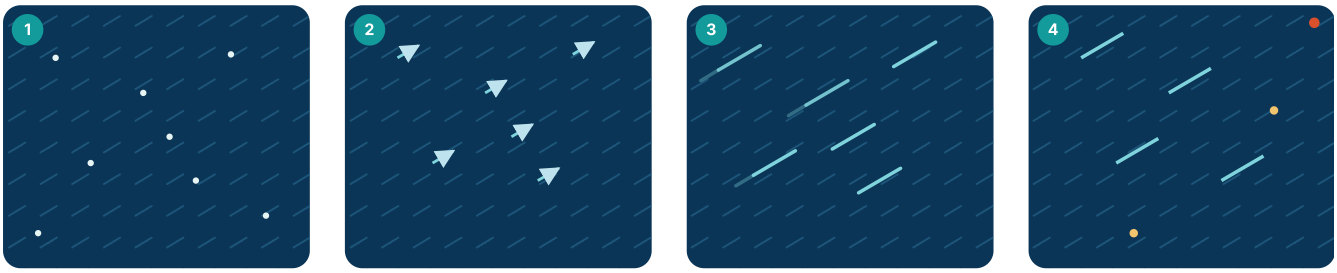
CHAPTER 11 · STORYBOARD

The map and the GPU particle engine

`MapManager` owns one MapLibre instance. Basemaps are either raster (Satellite, Blue Marble) or vector (Abyss, Bathymetry: OpenFreeMap tiles redrawn every frame for smooth zoom). Data layers register through `layerRegistry.ts` (22 static layers plus runtime-registered forecast layers).

How flowing currents are drawn

A vector field (u, v per cell) is fetched once and packed into a texture. A WebGL2 custom layer (`VectorFieldParticleLayer.ts`) then moves tens of thousands of particles on the GPU each frame.



Seed
Random positions written to a particle-state texture.

Advect
Fragment shader samples u,v at each particle; pos += v·dt·speed.

Draw trails
Previous frame faded by α, new segments drawn: streaks appear.

Respawn
Particles that age out, leave view or hit land reseed (yellow). Loop.

One loop ≈ 16 ms (60 fps). Speed maps to brightness and trail length; land is a NaN mask in the texture so particles never draw on continents.

Storyboard E. The particle loop, as used for currents, wind, Stokes drift, combined drift and forecast vector fields.

Notable engineering decisions

1. **No 3D terrain.** MapLibre drapes overlays onto the terrain mesh; on a bathymetric DEM that mesh is the seafloor, so SST rendered kilometres below the camera. Bathymetry uses a flat hillshade instead.
2. **ResizeObserver on the container.** MapLibre only listens to window resize; the dashboard's embedded map needed this or it rendered blank.
3. **Glyphs set once.** `setGlyphs()` mid-session triggers a style reload that drops sources added in the same tick.
4. **Source specs are cloned** before `addSource` because MapLibre mutates them and two map instances can coexist.
5. **4K export.** The map can render a true ~4K PNG of the current view.

CHAPTER 12 · STORYBOARD

Detectors: eddies, heatwaves, upwelling

12.1 Mesoscale eddies (services/eddies.py, eddy_tracking.py)

Detection reads the currents cache and computes the **Okubo-Weiss parameter** $W = s_n^2 + s_s^2 - \omega^2$ (normal strain, shear strain, relative vorticity). Cells where rotation dominates strain (W below a negative threshold) are labelled into connected components; each component is an eddy. Polarity is `sign(ω) == sign(latitude)` → cyclonic. Tracking links each new frame's eddies to the previous frame's using the Hungarian algorithm when matches are ambiguous.



Storyboard F. Eddy detection and tracking.

12.2 Marine heatwaves (services/heatwaves.py, heatwave_tracking.py)

Hobday et al. (2016): SST above the **seasonally varying 90th percentile** for **at least 5 consecutive days**. The percentile comes from a 30-year OISST climatology fitted with a leap-adjusted 1..366 day index (`services/climatology/`). Category = multiples of the (p90 - mean) gap: 1-2× moderate, 2-3× strong, 3-4× severe, 4×+ extreme. Headline statistics are taken over 60°S-60°N only, because ice-margin cells otherwise dominate.

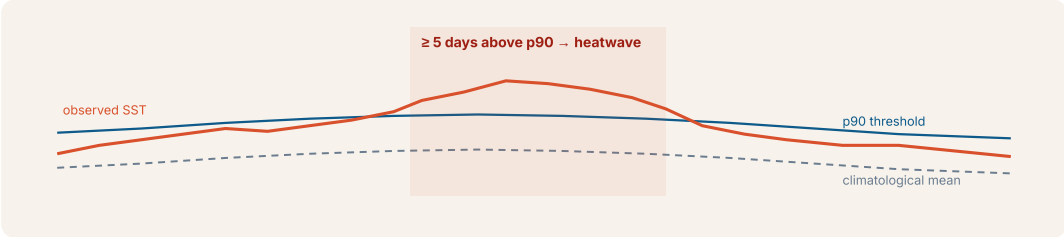


Figure 4. Heatwave definition. On 3 Oct 2026, EC2 logged 11,164 of 31,830 ocean cells (60°S-60°N) in heatwave: 35.1%.

12.3 Coastal upwelling (services/upwelling.py)

A **Bakun index** from wind stress: offshore Ekman transport along each coastal segment's normal. It is **corroborated** separately against a cold SST anomaly (the heatwave arithmetic with the sign reversed, using p10). The two are published as two separate claims, each with its own coverage and lag. A `sst_unavailable` state is distinct from "not corroborated".

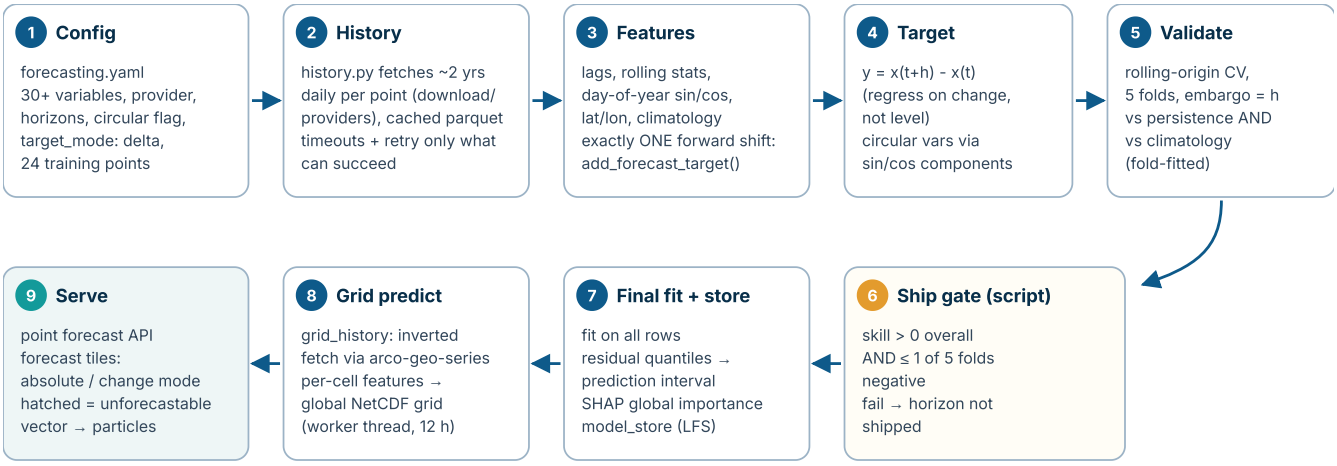
12.4 eDNA coverage (services/edna.py)

OBIS records with `DNADerivedData`, binned to geohash cells on a fixed log ramp. Coverage is a few hundredths of a percent of the ocean; the map states that rather than looking like a failed load.

CHAPTER 13 · PIPELINE

The forecasting engine

One framework for every variable. Adding a variable is a YAML block in `forecasting/config/forecasting.yaml` plus a training run. Models are **global** (one LightGBM per variable × horizon, trained on ~24 locations pooled) so they can be applied anywhere.



Open-Meteo-only variables can be forecast at points but never gridded; this is stated in the API rather than hidden.

Figure 5. Nine-step forecasting pipeline, from YAML to the forecast map.

Rolling-origin validation with an embargo

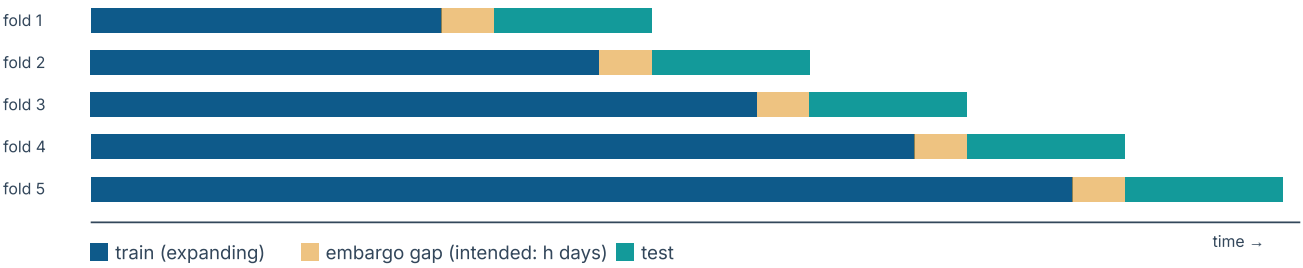


Figure 6. Expanding-window CV. Never random K-fold. The gap is meant to stop a training row's label at t+h from landing inside the test window. Part III (finding L2) shows the gap is counted in rows, not days, on the pooled frame.

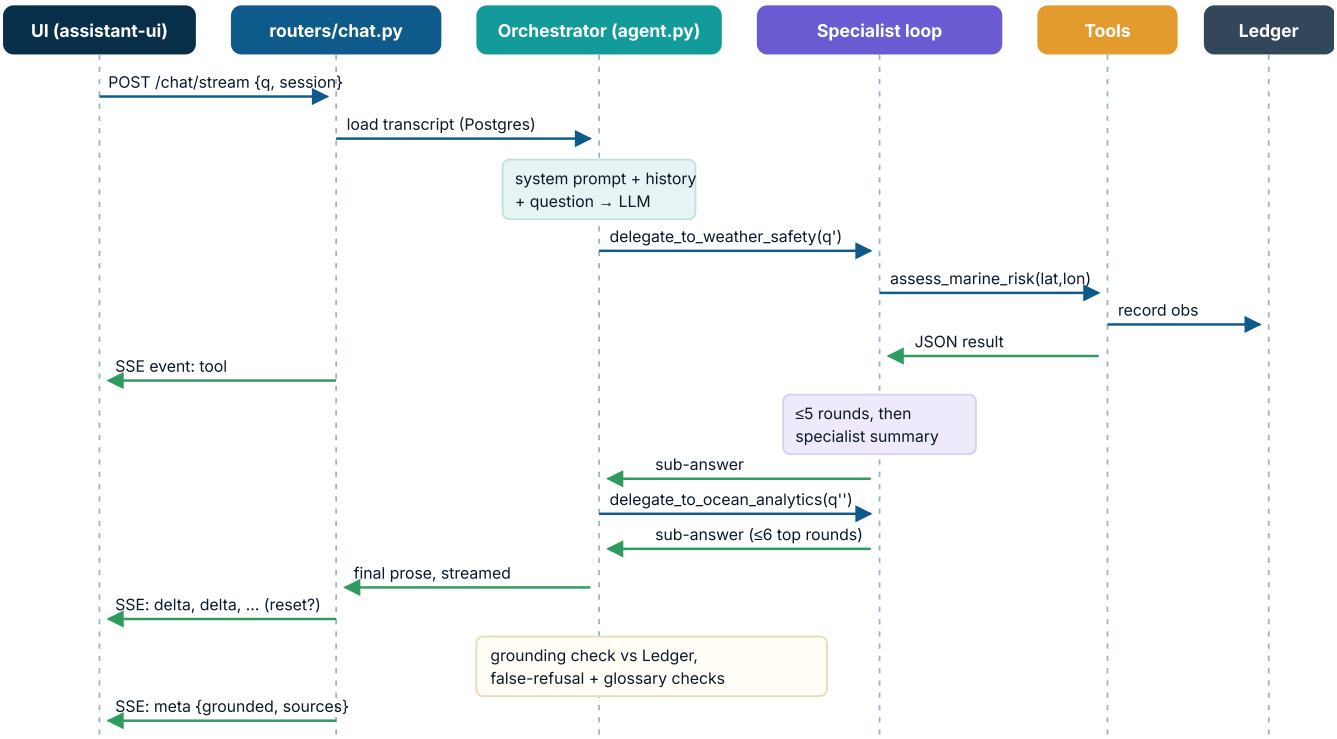
> The **skill score** is `1 - RMSE`$_{\texttt{model}}$ `/ RMSE`$_{\texttt{persistence}}$. A second skill against a fold-fitted climatology exists because persistence decays with horizon while a seasonal cycle does not: a model that only learned the season looks better and better against persistence as the horizon grows.

CHAPTER 14 · SEQUENCE

The Ocean Assistant (multi-agent)

A bounded tool-calling loop, not RAG. A top-level orchestrator delegates to four domain specialists. Each specialist is its own smaller loop over an allowlist of tools that wrap the platform's own services.

SPECIALIST	TOOLS
ocean_analytics	`list_available_variables, get_point_forecast, get_forecast_trend, get_global_ocean_summary, get_bloom_risk, get_fishing_habitat, find_fishing_zones, get_historical_series, analyze_variable_correlation, get_sst_anomaly`
weather_safety	`get_current_conditions, get_active_alerts, get_cyclone_alerts, get_severe_weather_alerts, get_tide_level, get_argo_profile, get_historical_series, assess_marine_risk`
geospatial_risk	`check_geofence, get_seafloor_depth, plan_safe_route, plan_drift_trajectory`
external_research	`web_search (Tavily), fetch_webpage, search_scientific_literature`



Sequence G. One assistant turn. The `grounded` verdict needs the finished text, so it can only ride the final `meta` event; the UI shows streaming text as unverified until then.

Key mechanics

1. **Provider-agnostic LLM.** `LLM_PROVIDER` selects Gemini, any OpenAI-compatible endpoint, or Ollama through LangChain; retry with backoff on transient errors.

2. **The stored transcript is the authority** on what was said (when Postgres is configured), not the browser's copy, so a client can't rewrite history.
3. **Delegation is traced.** The orchestrator's own restatement of each sub-task (`question` argument) is recorded with the chosen specialist: a cheap, honest "why it delegated".
4. **Multilingual by prompt.** Detect the question's language, answer in kind (ten Indian languages named). A glossary check flags domain terms that were likely mistranslated.
5. **Truncation is explicit.** If the loop exhausts its rounds still wanting tools, the response says `truncated: true` rather than returning a stale previous answer.
6. **Structured geometry.** Map-able results (routes, zones, drift paths) ride a `chat-geometry` layer so the embedded map can draw them; reopened chats restore maps, charts and provenance (commit e0b73c7).

CHAPTER 15

Grounding: how numbers are verified

After the answer is complete, `_ungrounded_numbers()` extracts every quantity in it and checks that each one appears in something the model was actually shown.

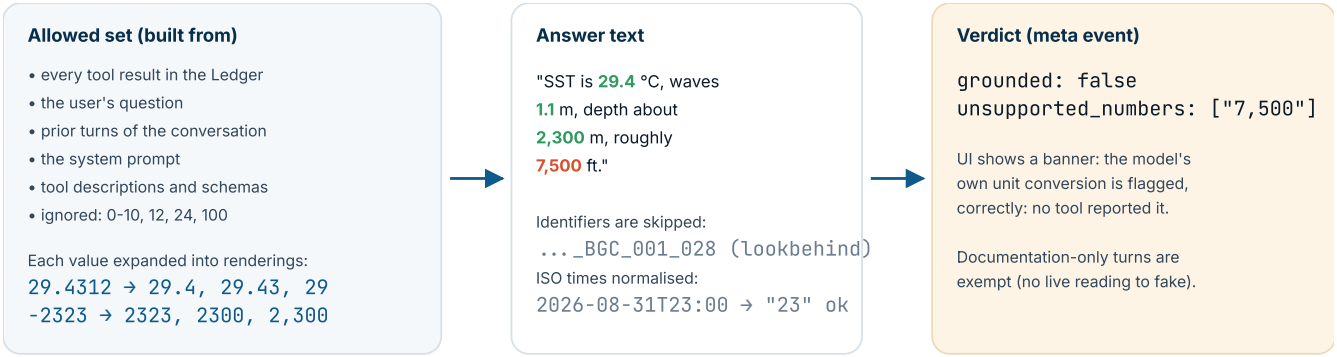


Figure 7. The grounding check. Two companion checks run on the same text: `_false_refusal` (the model says it "couldn't get" data that a tool did return) and `_untranslated_glossary_terms`.

> **Known weakness** (Part III, L11): because prior turns and the system prompt feed the allowed set, a number the model invented in an earlier turn, or any threshold in the prompt, passes the check when repeated later.

CHAPTER 16 · STORYBOARD

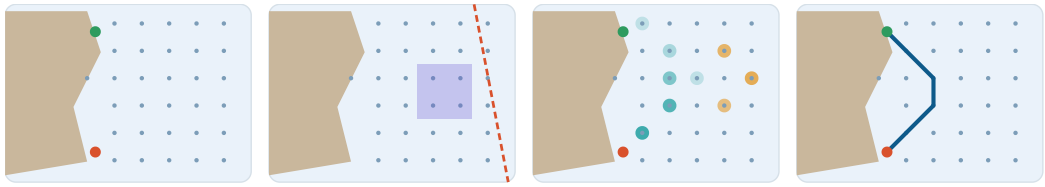
Safety tools: risk, routing, geofence, alerts

16.1 Deterministic risk verdict (services/marine_risk.py)

Four checks run concurrently: Open-Meteo current conditions, IMD CAP alerts covering the point, GDACS cyclones, and local geofencing. A fixed rule table reduces them so the same conditions always give the same `risk_level`. Escalation only goes up; a failed check goes to `could_not_verify`.

INPUT	MODERATE	HIGH	EXTREME
Wave height	≥ 1.5 m	≥ 2.5 m	-
Wind speed (10 m)	≥ 40 km/h (Beaufort 6)	≥ 62 km/h (gale)	-
IMD CAP alert covering the point	any other severity	"Severe"	"Extreme"
Active cyclone (last fix)	-	-	within 500 km
Geofence	≤ 20 km from IMBL, or inside an MPA	-	-

16.2 Hazard-aware A* route planning (services/routing.py)



1 · Grid + land mask
bbox + margin; 22 nodes on long side; GEBCO decides water/land

2 · Exclusions
nodes inside MPAs or ≤ 20 km of the IMBL removed; draft too deep

3 · Weight + expand
cost = km × (1 + wave m); orange = rough water; A* expands (teal)

4 · Path + simplify
edges sampled for land; path simplified to turn points

Heuristic = haversine distance (admissible: every edge costs at least its distance). An optional vessel draft removes shallow nodes and edges; speed and fuel range are applied after the search. Endpoints connect to every nearby water node whose straight segment stays in water, so a harbour on the coarse mask's edge still works.

Storyboard H. Route planning in four frames. Part III (L5-L7) covers gaps: unknown hazard is costed as calm, wind is ignored, and exclusion is checked at nodes only.

16.3 Geofencing (services/geofencing.py)

Pure local geometry, no network: India's EEZ (mainland + Andaman & Nicobar, Marine Regions MRGID 8480/8333, simplified to ~1,500 vertices, 137 interior holes kept), the 1974/1976 India-Sri Lanka treaty line (26 points), and 9 hand-curated MPAs as boxes around published centres. Returns inside/near/distance for each.

16.4 Cyclones and IMD alerts

GDACS event list (keyless), filtered to `iscurrent` tropical cyclones, distance from the point to each storm's *last reported fix* (not its forecast cone, because GDACS geometry endpoints did not respond within 120 s when probed). **IMD CAP 1.2**: RSS index → up to 20 CAP XML alerts → polygons/circles parsed with Shapely → point-in-area test within the onset/expires window.

16.5 Other point tools

Tides: INCOIS real-time tide-gauge sea level (measured, not a predicted tide table). **Drift trajectory**: particle advection through currents + Stokes drift + windage. **PFZ screening**: chlorophyll above the local median and SST in 24-30 °C, scored over a 0.5° candidate grid. **Argo**: nearest float profile.

CHAPTER 17

Dashboard, downloader, point brief

Ocean Intelligence Dashboard

1. **KPIs from scheduled caches** (never fetched per request), each with a three-way `ready / warming / unavailable` status and a sparkline from an in-process ring buffer.
2. **Provider health** shows which upstreams are live, stale or down.
3. **Alerts are threshold rules over real fields**, explicitly not issued warnings.
4. **Ocean Story** (`services/metrics/story.py`) computes the facts first, then phrases them; a verifier rejects sentences whose numbers are not in the computed block.
5. **Dossier pages** for each metric, the forecasting model (reproducibility, folds, reliability), buoys and satellite products. Series are decimated with a min/max envelope, never stride sampling, so peaks survive.
6. **Ocean heat content** is derived, not fetched: $\rho \cdot c_p \cdot \int T\, dz$ over Copernicus depth levels.

Universal Ocean Data Downloader (services/download/)

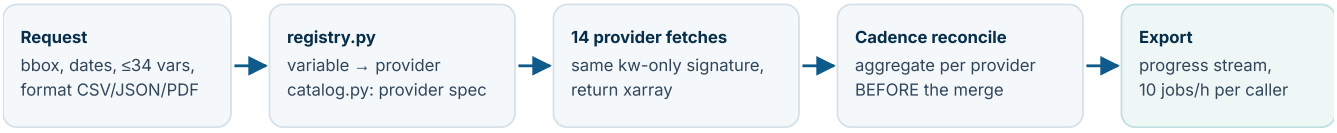


Figure 8. Downloader. Hourly, 3-hourly, daily and time-invariant sources are aggregated to the requested cadence per provider first, then merged.

Point brief (services/brief.py, brief_pdf.py)

Composition, not computation: every section reuses an existing endpoint (conditions, forecasts, nearest eddy, upwelling, heatwave, habitat, bloom risk, OBIS biodiversity) gathered concurrently, rendered on screen or as a ReportLab PDF. Each section is `available` or carries its reason.

CHAPTER 18 · STATE MACHINE

Alert watches (email)

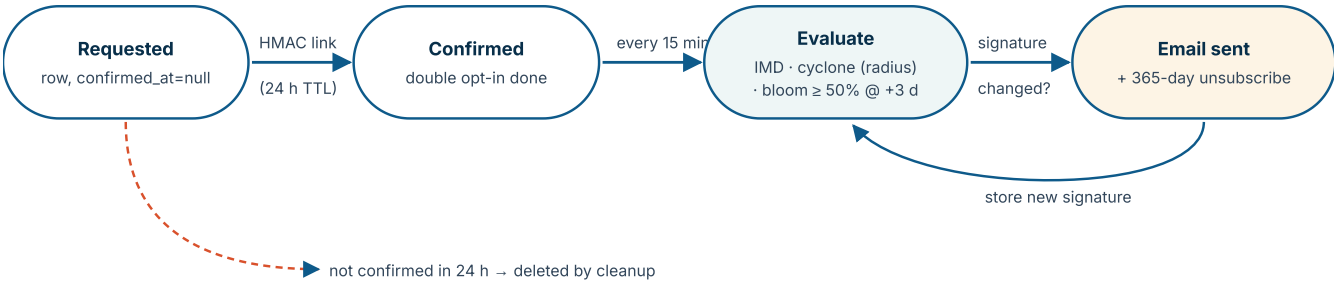


Figure 9. Watch lifecycle. The signature is the sorted set of active reasons (alert URLs, cyclone name, "bloom"), so the same condition is not emailed twice. "High waves" is a stated v1 scope cut.

Tokens are `base64url(id:purpose:expiry).hex(HMAC-SHA256)` with constant-time comparison; a confirm token cannot be used to unsubscribe. "My watches" is scoped by a random per-browser `client_id`. Creation is rate-limited (5/hour) because each spends an email.

CHAPTER 19 · PIPELINE

Offline machine learning

`machine_learning/` is a separate workspace with its own venv. The backend never imports it; it only serves the NetCDF grids and `manifest.json` that it exports.

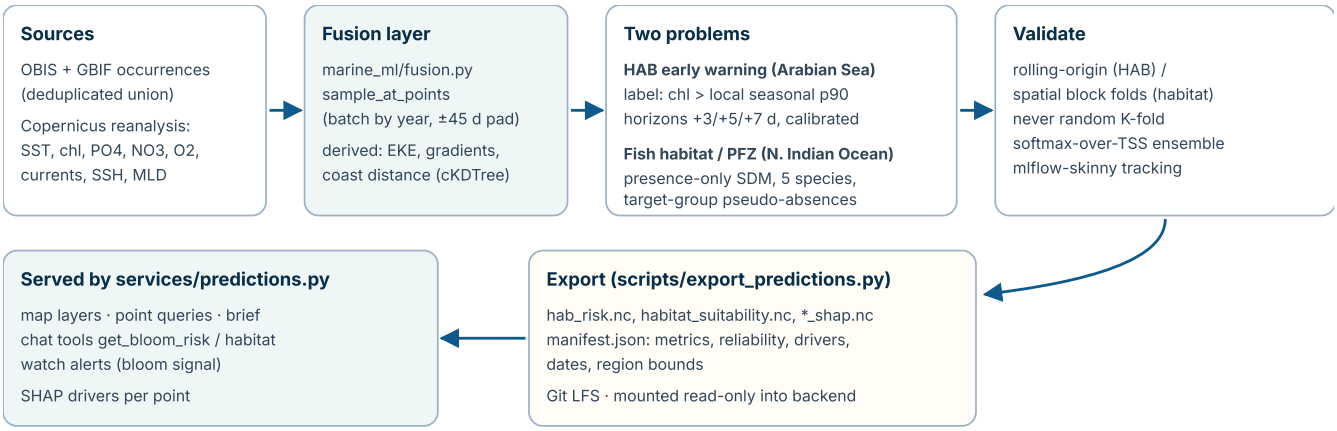


Figure 10. Offline ML. Two pipelines share one Marine Data Fusion Layer.

Published model quality (from manifest.json)

HAB HORIZON	PR-AUC	PERSISTENCE PR-AUC	ROC-AUC	BRIER	TSS
+3 days	0.661	0.511	0.942	0.038	0.751
+5 days	0.493	0.384	0.884	0.049	0.635
+7 days	0.362	0.309	0.841	0.057	0.564

Top drivers at +3 d: chlorophyll anomaly, z-anomaly, raw chlorophyll, current bloom state, chlorophyll change. Habitat model: TSS 0.76-0.90 across five spatial folds. **Habitat window ends 2013** because labels collapse after 2014 in both OBIS and GBIF; **the exported HAB grid covers 25 Nov - 24 Dec 2021** (see Part III, L1).

CHAPTER 20 · DEPLOYMENT

Deployment on AWS EC2

As found on 19 Oct 2026: instance `i-0c7686f701d6b0af1` ("Maris AI"), t3.xlarge (4 vCPU, 16 GB), Amazon Linux 2023, ap-south-1a, one 16 GB gp volume, security group `launch-wizard-1`.

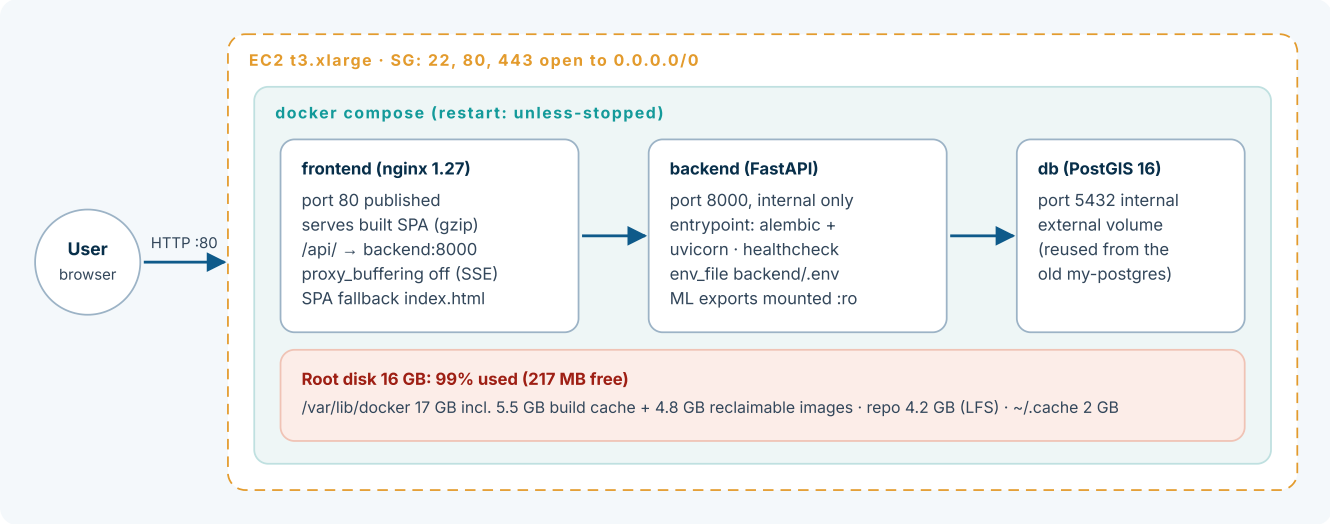


Figure 11. Production topology. The production-shaped nginx frontend and the internal-only backend exist only as uncommitted edits on the instance (Part III, D2).

Outbound dependencies at runtime

Copernicus Marine (S3/zarr), NOAA ERDDAP, NASA GIBS/Earthdata, Open-Meteo, GFW, AISStream websocket, GDACS, IMD CAP on S3, INCOIS, OBIS, Marine Regions, Ifremer GEBCO, the configured LLM provider, Tavily (if keyed) and Gmail SMTP.

CHAPTER 21

Technology stack reference

LAYER	TECHNOLOGY
Frontend	React 19, TypeScript 6, Vite 8 (Rolldown), Tailwind 4, Radix UI, TanStack Query 5, Zustand 5, MapLibre GL 6, custom WebGL2 layers, OGL, Recharts 3, uPlot, GSAP + Lenis + Framer Motion (landing), assistant-ui 0.15, oxlint
Backend	Python 3.11, FastAPI, Pydantic 2 + pydantic-settings, uvicorn, APScheduler, httpx/aiohttp/websockets, tenacity, loguru, SQLAlchemy 2 (asyncpg) + Alembic, ReportLab, Pillow, matplotlib
Scientific	numpy, pandas, xarray, netCDF4/h5netcdf, scipy, shapely, pyproj, numba, copernicusmarine
ML	LightGBM, scikit-learn, SHAP, Prophet baseline, mlflow-skinny (ML workspace)
LLM	LangChain core + Google GenAI / OpenAI-compatible / Ollama adapters
Data	PostgreSQL/PostGIS 16 (optional), NetCDF/parquet/JSON disk caches, Git LFS for models and exports
Ops	Docker multi-stage images, docker compose, nginx, AWS EC2 (ap-south-1)

~115k
LINES OF CODE

~100
API ENDPOINTS

17
SCHEDULED JOBS

25
ASSISTANT TOOLS

30+
FORECASTABLE VARIABLES

14
DOWNLOAD PROVIDERS

III

Audit report

Carried out 19 Oct 2026. The EC2 instance was started only to inspect it, then stopped again. Nothing was changed: findings are reported, not fixed.

CHAPTER 22

Version check: EC2 vs local vs GitHub

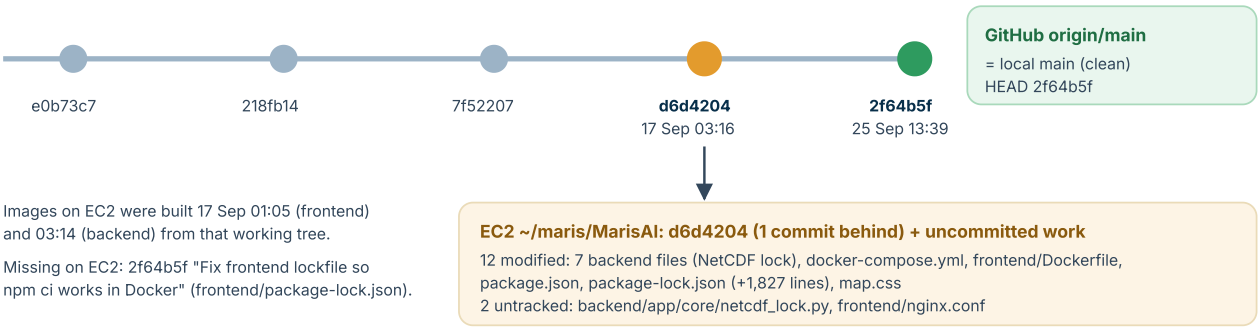


Figure 12. Version graph. Local and GitHub are identical and newest. EC2 is one commit behind and carries production changes that exist nowhere else.

LOCATION	COMMIT	STATE	NEWEST?
GitHub (VelvetGradient-26/MarisAI, main)	`2f64b5f`	Only branch is main; tag prototype-2026-07	✔️ newest
Local (~/Projects/maris_neomorphism/MarisAI)	`2f64b5f`	Clean, in sync with origin/main	✔️ same as GitHub
EC2 (i-0c7686f701d6b0af1)	`d6d4204`	14 uncommitted/untracked files, running containers built from them	✗ 1 behind, and diverged

So: not the same version. Pulling GitHub onto EC2 would conflict on `frontend/package-lock.json`. Redeploying from GitHub as-is would drop the HDF5 segfault fix and the production nginx frontend, bringing back the Vite dev server on port 5173 and port 8000 exposed.

CHAPTER 23

Deployment and infrastructure findings

CRITICAL wrong output reaching users or data loss · HIGH real defect, likely to bite · MEDIUM defect under specific conditions · LOW hygiene

HIGH D1 · Production fixes exist only on the EC2 disk

The HDF5 lock (`app/core/netcdf_lock.py` + 7 call sites), the multi-stage nginx frontend (`frontend/Dockerfile`, `nginx.conf`), the internal-only backend, the external DB volume and the `.map-root` `.map-container` CSS fix were never committed. The lock's own docstring records two production segfaults (`sig=11` in libhdf5) that it fixes. A rebuild from GitHub reintroduces them.

`EC2: ~/maris/MarisAI (git status: 12 M, 2 ??)`

HIGH D2 · Root volume is 99% full

16 GB volume with 217 MB free. Docker holds 17 GB, of which 5.5 GB is build cache and 4.8 GB unused images. The next image build, a forecast-grid rebuild (README: ~3 GB peak per variable) or a disk-cache write will fail with ENOSPC; atomic `.tmp` writes can fail and leave the cache stale.

`df -h /: 16G 16G 217M 99% · docker system df`

MEDIUM D3 · Accidental npm packages in the deployed frontend

`"20": "^1.0.0"` and `"nvm": "^0.0.4"` were added to `frontend/package.json` dependencies on EC2. This looks like `npm install nvm 20` run inside `frontend/` instead of `nvm install 20`. They are unrelated third-party packages now shipped in the build (supply-chain surface) and are part of the 1,827-line lockfile diff.

`EC2: frontend/package.json`

MEDIUM D4 · Watch emails link to localhost in production

`FRONTEND_BASE_URL` defaults to `http://localhost:5173` and is not set in the EC2 `backend/.env` or compose file. Every confirmation and unsubscribe link emailed from production points at the recipient's own machine, so no watch can be confirmed and no alert can be unsubscribed.

`app/core/config.py:21 · services/watch_alerts.py _confirm_url/_unsubscribe_url`

MEDIUM D5 · Credentials flagged as exposed, plus unused secrets

The EC2 `.env` carries a comment that the MongoDB Atlas password "was exposed in a chat transcript" and should be rotated. MongoDB and Google OAuth credentials remain in the file although auth was removed (docs/AUTH_REMOVAL.md). Values were not read during this audit. Confirm rotation and remove what is unused.

`EC2: backend/.env`

MEDIUM D6 · Rate limits are bypassable behind nginx

`client_key()` trusts the left-most `X-Forwarded-For`. nginx uses `$proxy_add_x_forwarded_for`, which appends to whatever the client sent, so any caller picks their own key per request. This defeats the download, insights, feedback and watch limits. Watch creation can then be used to send confirmation emails to arbitrary addresses at scale.

`services/rate_limit.py client_key · EC2 frontend/nginx.conf`

MEDIUM D7 · Chat endpoints have no rate limit

Download, insights, feedback, watch and metrics routes have limiters; `/api/v1/chat` and `/chat/stream`, the most expensive endpoints (up to 6 + 4×5 LLM calls per turn plus web search), have none. The instance is publicly reachable on port 80.

`routers/chat.py`

LOW D8 · Network exposure

SSH (22) is open to 0.0.0.0/0. 443 is open but nothing listens, so the app is HTTP only and the assistant, feedback and watch forms (emails) travel unencrypted.

`sg-0c2bd5f32efbdd067`

LOW D9 · NetCDF lock: two gaps in the EC2-only fix

(a) `oisst._open` / `fetch_recent` return a lazily opened dataset from inside the lock, so the actual HDF5 reads happen later, outside it. (b) `forecast_tiles._load_grid` takes the lock on the event-loop thread; while a background grid rebuild holds it for a large `to_netcdf`, every request stalls.

`EC2: services/climatology/oisst.py · services/forecast_tiles.py`

LOW D10 · Local workstation hygiene

`frontend/node_modules` is an empty root-owned directory left by the old compose anonymous volume, so `npm ci` fails locally with EACCES. A stale remote-tracking branch `origin/neumorphism-redesign` remains although it is gone from GitHub (`git fetch --prune`).

CHAPTER 24

Business-logic findings

CRITICAL L1 · Five-year-old bloom forecasts are presented as current

The exported HAB grid covers only **2021-11-25 to 2021-12-24**. `hab_slice()` defaults to the last exported day, so: (a) **watch alerts** email "Harmful algal bloom risk 62% at +3d" with no date, as if it were a live forecast; (b) the **point brief** shows "+3 days / +5 days / +7 days" risk rows with no date; (c) the chat tool returns the date, but nothing forces the model to say it. Since the grid never changes, any watched point above 50% gets an alert for a December 2021 condition. The map hook does label it historical.

`services/predictions.py hab_slice · services/watch_alerts.py:422 · services/brief.py _bloom_section · exports/manifest.json "dates"`

HIGH L2 · Forecast validation embargo is in rows, not days

`rolling_origin_splits` drops `gap = horizon` **rows** before each cutoff. The training frame pools ~24 locations sorted by time, so a 7-row gap is about a third of one day, not 7 days. Training rows whose labels (t+h) fall inside the test window survive, and one date can be split across train and test. Skill scores, the ship gate and the published model dossier are therefore optimistic. `chronological_split` has the same pattern.

`forecasting/evaluator.py:82-104, 120-130 · forecasting/trainer.py:354 (pooled sort)`

HIGH L3 · "Is it safe tomorrow?" is answered with today's conditions

`assess_marine_risk` takes only a coordinate and reads Open-Meteo *current* values. The tool description tells the model to prefer it for any safety question and never says it is "now only". A flagship problem-statement query ("safe to venture tomorrow morning") gets a deterministic verdict about the present, delivered as if it answered the question.

`services/marine_risk.py assess · services/chat/tools.py:850`

HIGH L4 · "Low risk" when nothing was verified

If the conditions, IMD and GDACS checks all fail, the level stays at its `"low"` default and the reason reads "no hazard condition crossed a fixed threshold in the checks that ran". Separately, when Open-Meteo answers but `wave_height` or `wind_speed` is null (common at coastal cells), that check counts as "checked" with nothing added to `could_not_verify`. Both contradict the module's own "a gap is not evidence of safety" rule.

`services/marine_risk.py _assess_conditions, assess`

HIGH L5 · Router prefers water it has no data for

When a hazard batch fails, nodes keep `wave_height_m=None` and `_edge_cost` uses factor 1.0, the same as perfectly flat sea and cheaper than any measured cell. A* is therefore drawn into unmeasured water. `hazard_level` is computed only from known waves, so a route crossing unknown cells is reported "calm".

`services/routing.py _edge_cost, _fetch_hazard_grid, plan_route`

HIGH L6 · Router ignores wind

Wind speed is fetched for every node and returned per waypoint, but it plays no part in the edge cost or the route's `hazard_level`. A gale with small waves (offshore wind near the coast) is routed and labelled "calm", while `marine_risk` would call the same point "high".

`services/routing.py _edge_cost, _hazard_level`

MEDIUM L7 · Routes can cut through excluded zones between nodes

MPA and IMBL exclusion is tested only at grid nodes; edges are tested only for land/depth. Grid spacing reaches 0.4° (~44 km; ~62 km diagonally), larger than the 40 km IMBL band and much larger than the smaller MPA boxes (Point Calimere is 0.1° wide). The docstring's "structurally cannot cross" claim does not hold for long routes.

`services/routing.py _is_geofence_excluded, adjacency loop`

MEDIUM L8 · Watch alerts repeat after a feed outage

A failed IMD or GDACS fetch is logged and skipped, producing an empty signature. That is stored as "no alerts", so when the feed recovers the unchanged alert looks new and is emailed again. An alert set shrinking (A+B → A) also triggers a fresh "conditions changed" email.

`services/watch_alerts.py _evaluate_one, evaluate_and_notify`

MEDIUM L9 · IMD alerts can be silently missed

Only the first 20 RSS items are read. A CAP alert without `expires` is treated as inactive. An alert described only by `areaDesc`/geocode (no polygon or circle) can never cover a point. Individual CAP fetch failures are dropped, and the check still reports as successfully "checked". A naive timestamp would raise a TypeError on comparison and fail the whole check.

`services/severe_weather.py _Alert.is_active, covers, _fetch_alerts`

MEDIUM L10 · "Nearest fishing zone" is not the nearest

`find_zones` ranks by score, then by highest chlorophyll; distance is neither used nor returned. The problem statement's "Where is the nearest PFZ?" can return a farther cell. Tie-breaking on maximum chlorophyll also favours turbid coastal or blooming water, which is not where pelagic fish are expected.

`services/pfz.py find_zones`

MEDIUM L11 · Grounding check can be laundered

The allowed set includes all prior turns, including the assistant's own earlier (possibly unverified) answers, and the full system prompt with its thresholds (1.5, 2.5, 24, 30, 40, 62 ...). An invented number from turn 1 passes in turn 2, and a fabricated "SST 24 °C" passes because 24 is in the prompt. Answer-side rounding also accepts values within ±0.05 of any real one (2.64 passes against a real 2.58).

`services/chat/agent.py answer (shown = ...), _ungrounded_numbers`

MEDIUM **L12 · Heatwave tracking deviates from Hobday and resets on restart**

(a) No 2-day gap-merging rule: a single cooler day ends an event and restarts onset and duration. (b) Runs count array positions, not calendar days, so a missing day in the OISST record is bridged as consecutive. (c) Tracked state lives only in memory, so the "true, uncapped duration" resets on every restart (EC2 was stopped for four weeks).

`services/heatwave_tracking.py advance`

MEDIUM **L14 · ML environment does not reproduce from requirements.txt**

`sqlalchemy ⩾ 2.0` now resolves to 2.1.5, which removed `FallbackAsyncAdaptedQueuePool`; the pinned `mlflow-skinny==3.6.0` imports it. A fresh install fails all 18 tracking and forecasting-ingestion tests, and tracking would silently not record (it is designed never to fail a run).

`machine_learning/requirements.txt:58-59 · tests/test_tracking.py, tests/test_ingest_forecasting.py`

LOW **L15 · Background KPI persistence leaks unhandled task errors**

`dashboard/history._persist` is fire-and-forget (`loop.create_task`) and catches only `SQLAlchemyError`. A refused connection raises `OSError`, which escapes as "Task exception was never retrieved" instead of the intended warning. Seen many times in the test run.

`backend/services/dashboard/history.py:102-111`

LOW **L13 · Documentation drift**

`docs/features/ocean-assistant.md` says the assistant has "three specialists"; the code (and README) have four, including `external_research`. The graphify knowledge graph was built from commit 90ae239e and is stale against HEAD.

CHAPTER 25

Test, lint and build results

CHECK	RESULT	NOTES
Frontend lint (oxlint)	✔️ clean	Run on a copy, because local node_modules is root-owned
Frontend typecheck (tsc -b)	✔️ 0 errors	
Frontend build (vite build)	✔️ built in 1.6 s	Warns: chunks over 500 kB (esri 939 kB, index 567 kB, AnalyticsGrid 362 kB)
Backend tests (pytest, 1st run with -x)	✖ 1 failed / 89 passed before stop	`test_chat_store.py` needs Postgres; it only skips when DATABASE_URL is unset, so a configured-but-down DB errors instead of skipping
Backend tests (full run)	879 passed / 16 failed	All 16 failures are "connection refused" to Postgres on 5432 (chat_store, dashboard_history_db, watch_alerts): no local DB was running. The skip guard tests whether DATABASE_URL is *set*, not whether the DB is *reachable*.
ML tests (machine_learning/.venv)	108 passed / 18 failed	Fresh venv from requirements.txt. Every failure is an ImportError: see L14.

Recommended order of attention

1. **Commit the EC2 work** (minus the stray `20` / `nvm` packages), rebase onto 2f64b5f, and free disk space on the instance before any rebuild.
2. **Stop presenting the 2021 HAB grid as current** in watch emails and the brief (L1).
3. **Make "unknown" never read as "safe"** in marine_risk and routing (L4, L5), and add wind to routing (L6).
4. **Re-run forecast validation with a date-based embargo** and re-check which horizons still pass the ship gate (L2).
5. **Add a time dimension to the risk verdict**, or have it say plainly that it describes "now" (L3).
6. Then the medium items: rate-limit keying and a chat limit, FRONTEND_BASE_URL, alert-signature handling on feed failure, CAP parsing gaps, PFZ distance, the grounding allowed-set.

Method: EC2 state via SSH (read-only commands) after `start-instances`, followed by `stop-instances`; versions via `git` on all three; code review by reading the safety-critical services (marine_risk, routing, geofencing, cyclones, severe_weather, pfz, watch_alerts, watch_tokens, rate_limit, chat agent grounding, heatwave tracking, forecasting evaluator/trainer, predictions) against the docs; automated checks as listed above. Not covered line by line: frontend components, downloader providers, dashboard metric services and the ML training code.